

Surgical Phase-Aware Reinforcement Learning for Autonomous Laparoscopic Camera Control

ID4900 Undergraduate Research Project Report

Indian Institute of Technology Madras



Mayank Chandak

Roll No: ME22B224

B.Tech, Department of Mechanical Engineering
Indian Institute of Technology Madras

Supervisor: Dr. Nirav Patel

Department of Engineering Design
Indian Institute of Technology Madras

INSPIRE Lab - Intelligent Systems Procedures and medical Interventions Research Lab
Chennai, India

Academic Year: 2025–2026

This work has been submitted to IROS 2026 as:

“Surgical Phase-Aware Reinforcement Learning for Autonomous Laparoscopic Camera Control”

Abstract

Robotic-assisted laparoscopic surgery depends critically on effective endoscopic camera control to maintain clear, stable, and informative visualization of surgical tools and anatomical structures. In current clinical practice, camera motion is largely governed manually by a surgeon or assistant, increasing cognitive workload and frequently resulting in suboptimal framing, particularly during complex multi-instrument procedures. Existing autonomous camera approaches are predominantly reactive, relying on visual servoing or fixed geometric objectives that do not adapt to the procedural context of a surgical workflow.

This report presents a complete description of the design, implementation, and evaluation of a **phase-aware reinforcement learning framework** for autonomous laparoscopic camera control. The camera is modeled as a four-degree-of-freedom pivoted endoscope subject to the Remote Center of Motion (RCM) kinematic constraint at the trocar insertion point. The learned control policy conditions jointly on RGB-D visual observations, instrument and camera velocity cues, and an explicit surgical phase signal. A phase-conditioned reward design enables the agent to adapt centering, zoom, and motion smoothness objectives dynamically across the distinct visualization priorities of the Grasp, Lift, and Touch phases of a laparoscopic cholecystectomy.

The entire system is implemented within a custom physics-based SOFA simulation environment built upon the LapGym framework, incorporating deformable tissue, articulated multi-instrument interaction, and stochastic human-like instrument noise. The framework is evaluated against two baselines: a phase-blind RL ablation and a weighted analytic heuristic controller with privileged geometric access. Results across 50 stochastic evaluation episodes demonstrate that the Phase-Aware policy achieves superior centering accuracy (0.59 vs. 0.42 for the oracle heuristic), near-perfect visibility coverage (0.99), and significantly reduced motion jitter (0.34 vs. 1.02 for phase-blind RL). These findings are submitted as a full paper to IEEE/RSJ IROS 2026.

Keywords: Surgical robotics, reinforcement learning, laparoscopic camera control, visual servoing, remote center of motion, surgical phase awareness, SOFA framework, proximal policy optimization.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Nirav Patel, for his guidance, patience, and intellectual generosity throughout this project. His insights into surgical robotics, feedback on reward design, and encouragement during the many debugging cycles were indispensable.

I am grateful to the INSPIRE Lab at IIT Madras for providing the computational resources and collaborative environment that made this work possible. I also thank the authors of the LapGym and sofa_env frameworks for their exceptional open-source contributions to surgical simulation research; this project would not have been feasible without their work.

Contents

Abstract	ii
Acknowledgements	iii
1 Introduction	1
1.1 Motivation and Clinical Context	1
1.2 Problem Statement	1
1.3 Contributions	2
2 Background and Related Work	3
2.1 Autonomous Surgical Camera Control	3
2.2 Surgical Phase Recognition	3
2.3 Data-Driven Camera Control	4
2.4 Reinforcement Learning in Surgical Robotics	4
2.5 Cinematic Motion Principles in Robotics	5
2.6 Surgical Task Planning	5
3 Simulation Environment	6
3.1 Overview	6
3.2 Anatomical Models	7
3.2.1 Liver	7
3.2.2 Gallbladder	7
3.2.3 Point of Interest (POI)	7
3.3 Instrument Models	7
3.3.1 Gripper	7
3.3.2 Cauter	8
3.3.3 Pivoted Camera	8
3.4 Scene Rendering and Dual-Camera Architecture	8
3.5 Contact and Collision Modeling	9
3.6 Task Structure and Phase Logic	9
3.7 Environment Reset and Randomization	10
4 Method	11

4.1	Markov Decision Process Formulation	11
4.2	Observation Space	11
4.2.1	Visual Observation I_t	12
4.2.2	Velocity Observations $\dot{\mathbf{q}}$	12
4.2.3	Phase Encoding p_t	12
4.3	Action Space	13
4.4	Remote Center of Motion Constraint	13
4.5	Scripted Tool Policy	13
4.6	Stochastic Instrument Motion Model	14
4.7	Phase-Conditioned Reward Design	15
4.7.1	Instrument Tip Point Projection	15
4.7.2	Phase-Dependent Attention Weights	15
4.7.3	Centering Score and Visibility	16
4.7.4	Zoom Reward	16
4.7.5	Motion Jitter Penalty	17
4.7.6	Collision Penalty	17
4.8	Policy Architecture and Training	17
5	Implementation	19
5.1	Code Architecture	19
5.2	Base Environment: <code>base_env.py</code>	19
5.3	Camera Environment: <code>camera_env.py</code>	20
5.4	Heuristic Baseline	20
5.5	Training Pipeline: <code>train.py</code>	21
5.6	Evaluation Pipeline	21
5.6.1	<code>evaluate.py</code> - RL Policy Evaluation	21
5.6.2	<code>evaluate_heuristic.py</code> - Heuristic Evaluation	21
6	Results and Analysis	22
6.1	Evaluated Controllers	22
6.2	Evaluation Setup	22
6.3	Aggregate Performance	22
6.4	Phase-Wise Centering Analysis	24
6.5	Temporal Stability	25
6.6	Statistical Significance	25
6.7	Episode Return and Outcomes	25
6.8	Zoom Regulation Analysis	26
7	Discussion and Limitations	27
7.1	Key Findings Summary	27

7.2	Ablation: Contribution of Phase Signal	27
7.3	Comparison with Prior Work	28
7.4	Simulation-Only Evaluation	28
7.5	Oracle Phase Signal	28
7.6	Limitations of the Reward Design	29
7.7	Extensions and Future Work	29
8	Conclusion	30
A	Network Architecture and Hyperparameters	32
B	Reward Component Weights	33
C	Stochastic Noise Model Parameters	34
D	Simulation Constants and Scene Parameters	35
E	Worked Reward Calculation Example	36

List of Figures

3.1	The physics-based SOFA simulation environment. Deformable gallbladder (yellow) rests on the rigid liver (red). The Gripper (blue), Cauter (green), and pivotized endoscopic camera (orange) enter through separate trocar insertion points. The fixed overhead camera (top-right, used for scripted tool logic) and the pivotized camera (agent-controlled) are both rendered each step.	6
4.1	System Overview: The reinforcement learning agent receives visual observations, instrument and camera velocities, and phase information; outputs 4-DOF velocity commands to the pivotized camera; and receives a phase-conditioned reward.	11
5.1	Project repository layout for reproducibility.	19
6.1	Comparative analysis of camera control performance across the three surgical phases. (a) Grouped bar chart of mean centering scores for Gripper, Cauter, and POI per phase. Phase-Aware RL (blue) consistently achieves the highest accuracy by dynamically adapting tracking priorities. (b) Temporal traces of centering scores with ± 1 std shading over 50 episodes. Vertical dashed lines indicate phase transitions.	24

List of Tables

6.1	Quantitative camera control performance (mean over 50 episodes). $\uparrow$ higher is better; $\downarrow$ lower is better.	23
A.1	PPO training hyperparameters	32
A.2	CNN encoder architecture (NatureCNN)	32
B.1	Reward function parameters	33
C.1	Stochastic instrument motion noise parameters	34
D.1	Simulation and scene parameters	35

Chapter 1

Introduction

1.1 Motivation and Clinical Context

Robotic-assisted laparoscopic surgery has transformed minimally invasive procedures by offering enhanced dexterity, tremor filtering, and high-definition three-dimensional visualization. Procedures such as laparoscopic cholecystectomy - the surgical removal of the gallbladder - are among the most commonly performed minimally invasive surgeries worldwide, with millions of cases annually. Despite the advanced robotic infrastructure available in platforms such as the da Vinci Surgical System, one aspect of the workflow remains surprisingly manual: the control of the endoscopic camera.

In the vast majority of laparoscopic procedures, the camera is either held by a surgical assistant following verbal commands from the operating surgeon, or it is stabilized by a mechanical arm and repositioned manually between steps. Both approaches suffer from well-documented limitations. Human assistants introduce latency between the surgeon's instruction and the camera response, may drift due to fatigue, and occasionally obscure the operating field. Mechanical holders are static and unable to track dynamic instrument motion. The result is that surgeons frequently divert attention from the primary task to manage the camera, increasing cognitive load and the risk of error.

The development of an autonomous camera controller - one capable of reliably maintaining informative views of the operative field - has therefore been a long-standing goal of surgical robotics research. However, designing such a controller is non-trivial. The optimal camera position varies continuously as instruments move, as anatomical structures deform, and as the task itself transitions through distinct procedural phases. A controller that is well-suited to the initial instrument grasping phase may provide poor visualization during fine dissection.

This work addresses precisely this limitation by developing a reinforcement learning (RL) framework that is explicitly conditioned on the current surgical phase.

1.2 Problem Statement

Formally, the problem is: given a physics-based simulation of a laparoscopic cholecystectomy with multiple controlled instruments, learn a continuous 4-DOF camera control

policy that maximizes the visibility, centering, zoom quality, and motion smoothness of the endoscopic view throughout the procedure, adapting dynamically to phase-specific visualization requirements.

The camera is constrained by the RCM constraint to pivoting motions about a fixed trocar insertion point, as in all real laparoscopic settings. The policy must not rely on privileged pose information at inference time - it must operate from visual observations and velocity cues only.

1.3 Contributions

The main contributions of this project are:

1. **A phase-aware RL formulation** for autonomous laparoscopic camera control, making the first systematic investigation into the use of procedural phase as a closed-loop control signal for camera motion.
2. **An RCM-constrained 4-DOF pivoted camera policy** implemented in a physics-based SOFA simulation of laparoscopic cholecystectomy, including full collision modeling between the camera and surgical instruments.
3. **A novel phase-conditioned dense reward design** that provides a continuous, non-trivial learning signal for cinematic visual quality (centering, zoom, jitter, and collision), explicitly avoiding the limitations of binary task-completion rewards and adapting across the Grasp, Lift, and Touch phases.
4. **A stochastic instrument motion model** incorporating human-like noise (correlated drift, high-frequency tremor, distance-dependent attenuation) to expose the camera policy to realistic motion variability during training.
5. **A weighted analytic heuristic baseline controller** implementing inverse pivot kinematics with phase-dependent target weighting, providing a strong oracle geometric reference.
6. **Comprehensive quantitative evaluation** across 50 episodes against both the analytic baseline and a phase-blind RL ablation, with per-phase analysis and temporal stability metrics.

Chapter 2

Background and Related Work

2.1 Autonomous Surgical Camera Control

The problem of autonomous camera control in robot-assisted surgery has attracted increasing research attention over the past decade. The earliest systematic approaches treated camera control as a visual servoing problem: the camera is commanded to track a designated point of interest (POI) or instrument tip by computing proportional-derivative control signals from the pixel error between the target and the image center [1, 2].

While effective under controlled conditions, pure visual servoing has fundamental limitations. It is inherently reactive, meaning it responds only to current error and has no model of how error will evolve. In multi-instrument scenarios where two or more tools compete for attention, visual servoing requires an explicit arbitration rule - typically a fixed priority ordering or a centroid of active tools - which may not be appropriate across different stages of the procedure. Additionally, proportional control reacts directly to high-frequency tremor in instrument motion, resulting in camera jitter that increases surgeon visual fatigue [3].

Several geometric optimization approaches have been proposed to handle multi-instrument visibility [4]. These methods define a composite objective over instrument positions projected into the image plane, optimizing for metrics such as the number of visible instruments, the bounding box coverage, or the weighted centroid error. Such methods can be solved efficiently in closed form but remain static in their optimization objective - the same weights and constraints apply regardless of what step of the surgery is being performed.

2.2 Surgical Phase Recognition

Surgical phase recognition has emerged as a mature subfield of computer-assisted intervention, with deep learning models achieving high accuracy on benchmark datasets for cholecystectomy and other procedures [5–7]. Architectures combining convolutional feature extraction with temporal modeling (LSTMs, transformers) are now capable of identifying phases such as preparation, clipping, dissection, and extraction from endoscopic video in real time.

Phase recognition has been widely applied for workflow analysis, surgical skill assessment, and operating room efficiency optimization. However, despite the availability of reliable real-time phase signals, their use as a direct input to closed-loop robotic control has received very little attention. Most surgical robotics controllers continue to operate with static objectives, implicitly assuming that visualization requirements do not change across the procedure.

This work explicitly bridges the gap between phase recognition as an analytical tool and phase conditioning as a control strategy.

2.3 Data-Driven Camera Control

Wagner et al. [8] proposed learning camera movement from expert surgical video, training a convolutional network to predict camera velocity from endoscopic observations. While this approach introduces implicit context awareness through temporal video features, the learned policy is not explicitly conditioned on procedural stage and cannot distinguish between Grasp and Touch phase requirements. Furthermore, imitation learning requires large annotated datasets of expert camera control, which are difficult and expensive to obtain.

Eslamian et al. [9] explored position-based camera control from structured surgical annotations. These methods suffer from similar annotation burden and static objective limitations.

2.4 Reinforcement Learning in Surgical Robotics

Reinforcement learning has been applied to a growing range of surgical robotics tasks, including suturing, tissue manipulation, needle insertion, and endoscope navigation [10–12]. The LapGym framework [13], upon which this project is built, provides a standardized collection of SOFA-based simulation environments specifically designed for RL research in laparoscopic surgery.

Prior RL-based camera controllers for surgical robotics typically formulate the problem with sparse task-completion rewards, which are difficult to optimize for continuous quality objectives. Motion smoothness and stability are rarely incorporated into the reward function. The RCM constraint, which is a hard physical requirement for any real laparoscopic camera, is often ignored in simulated settings, limiting the practical relevance of results.

This project addresses all three of these gaps: a core contribution is the formulation of a unique, dense reward signal designed specifically for cinematic visual quality. Because good camera control is about continuous framing rather than reaching a singular goal state,

this approach bypasses the limitations of binary 1/0 rewards. Furthermore, it incorporates motion stability terms and enforces the RCM constraint throughout.

2.5 Cinematic Motion Principles in Robotics

Outside the medical domain, principled formulations of visual framing objectives have been developed for aerial cinematography and autonomous videography [14]. These works formalize criteria such as subject centering, smooth motion, zoom regulation, and occlusion avoidance into optimization objectives. This project is inspired by these ideas, translating cinematographic quality metrics (centering, zoom, jitter) into a dense RL reward suitable for the surgical setting.

2.6 Surgical Task Planning

High-level surgical task planning methods have been proposed that reason about phase sequencing and instrument coordination at the symbolic level [15, 16]. These methods are complementary to the approach taken here: they determine *what* the surgical workflow requires at a macro level, while the phase-aware camera controller determines *how* to maintain appropriate visualization during each phase.

Chapter 3

Simulation Environment

3.1 Overview

The simulation environment is built using the SOFA Framework (v24.06.00) [17], a widely used open-source platform for real-time physically-based simulation of biological tissue and surgical instruments. The environment extends the LapGym framework [13] and is implemented as a Gymnasium-compatible environment class.

The scene models a laparoscopic cholecystectomy involving three instruments: a grasping forceps (Gripper), an electrocautery hook (Cauter), and the pivoted endoscopic camera. The anatomy consists of a deformable gallbladder attached to a rigid liver model. A visual target sphere (Point of Interest, POI) is placed on the gallbladder surface to represent the dissection target.

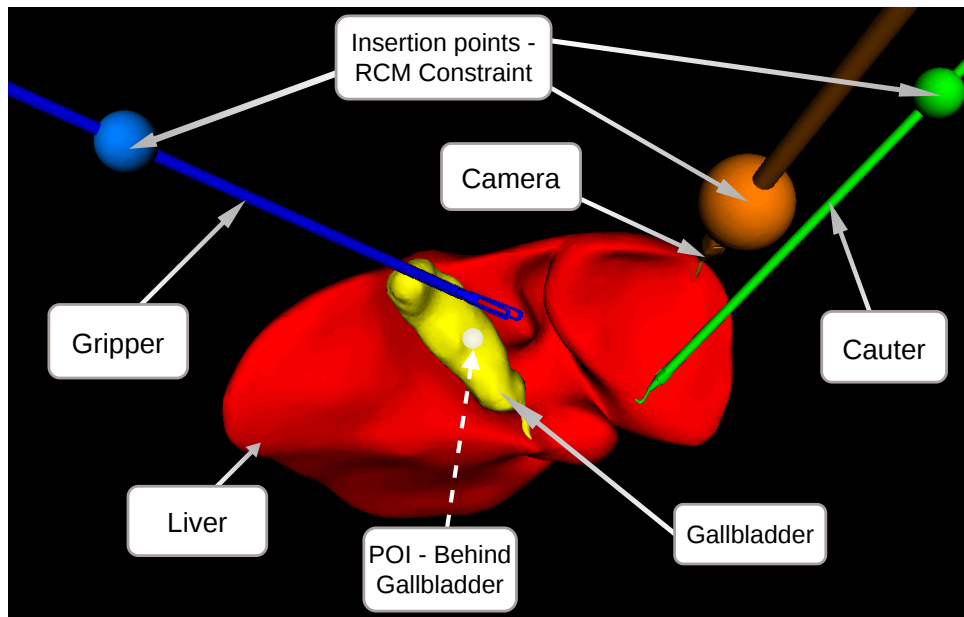


Figure 3.1: The physics-based SOFA simulation environment. Deformable gallbladder (yellow) rests on the rigid liver (red). The Gripper (blue), Cauter (green), and pivoted endoscopic camera (orange) enter through separate trocar insertion points. The fixed overhead camera (top-right, used for scripted tool logic) and the pivoted camera (agent-controlled) are both rendered each step.

3.2 Anatomical Models

3.2.1 Liver

The liver is modeled as a rigid body with a volumetric tetrahedral mesh for FEM-based collision response and a separate surface mesh for visual rendering. It acts as a fixed reference structure and collision partner for the instruments.

3.2.2 Gallbladder

The gallbladder is a deformable elastic body modeled using the SOFA FEM pipeline with a volumetric mesh (VTK format) and separate collision and visual surface meshes. A region-of-interest bounding box identifies the infundibulum (the graspable region). Gallbladder deformation is driven by grasping forces from the Gripper and by gravity ($g = 9.81 \text{ m/s}^2$, applied along the $+y$ axis in SOFA's coordinate convention).

A mesh-based overlap detection region monitors the degree to which the gallbladder penetrates the liver surface, which is used in the Lift phase logic to determine when the gallbladder has been sufficiently elevated.

3.2.3 Point of Interest (POI)

The POI is a small visual sphere placed on the gallbladder surface. Its 3D position is randomized at each episode reset (within a defined range) when `randomize_poi_position=True`. The POI's color (bright white/yellow) is chosen to be visible against the tissue textures, and its in-image presence is determined by a brightness threshold applied to a patch of the rendered image around its projected pixel coordinates.

3.3 Instrument Models

All three instruments (Gripper, Cauter, Camera) follow the standard LapGym pivotized rigid object model. Each instrument is parametrized in a four-dimensional state space $[\theta_p, \theta_t, \theta_s, d]$ (pan, tilt, spin angles in degrees, and insertion depth in mm), with a fixed Remote Center of Motion (RCM) pose defining the trocar location.

3.3.1 Gripper

The Gripper is a single-action laparoscopic forceps with a shaft and an articulated jaw. It has a fifth state dimension for jaw opening angle (0° = fully closed, 90° = fully open). Grasping is implemented through a spring-based force field in SOFA: when the jaw is sufficiently closed around gallbladder surface mesh vertices, a constrained spring system

is activated. Grasp quality is tracked by the number of active springs and their attachment to the infundibulum.

RCM pose: $[-160, 30, -200]$ mm, orientation $[0^\circ, 20^\circ, 45^\circ]$.

State limits: Pan $[-75^\circ, 75^\circ]$, Tilt $[-40^\circ, 75^\circ]$, Spin $[-1000^\circ, 1000^\circ]$, Depth $[12, 300]$ mm.

3.3.2 Cauter

The Cauter is a dissection electrode. It has a binary activation state (0 = inactive, 1 = active) as its fifth state dimension. Activation is used in the Touch phase to simulate cauterization when the cauter tip is within the goal tolerance of the POI.

RCM pose: $[-150, -200, 0]$ mm, orientation $[0^\circ, 90^\circ, 180^\circ]$.

State limits: identical to the Gripper.

3.3.3 Pivoted Camera

The pivoted camera is a four-DOF instrument (no jaw or activation). It uses the same pivot kinematic model as the Gripper and Cauter. Importantly, the camera object in SOFA is both a physical collision body (with a sphere-based collision model along its shaft) and a rendering camera. At each step, the environment renders the scene from the camera's current viewpoint and returns the result as the observation.

RCM pose: $[-155, -180, -260]$ mm, orientation $[0^\circ, 90^\circ, 180^\circ]$.

Initial PTSD state: $[40^\circ, 40^\circ, 400^\circ, 20\text{mm}]$.

Vertical field of view: 45° . Z-near: 20 mm, Z-far: 1500 mm.

3.4 Scene Rendering and Dual-Camera Architecture

A key architectural decision in this environment is the **dual-camera setup**:

- **Fixed overhead camera:** A static camera positioned at a predetermined overhead viewpoint. This camera renders a fixed reference view of the scene. It is used internally by the scripted tool policy to determine whether the POI target is visible (needed for the Lift phase logic and reactive lift control). Its viewpoint is never changed by the agent.
- **Pivoted camera:** The camera controlled by the RL agent. This camera is both a physical object in the scene (with collision geometry) and the observation source for the policy. At each step, the rendered image from this camera is the agent's observation.

This design ensures that the scripted tools always have access to reliable target visibility information regardless of where the RL agent points the endoscope. It prevents a pathological failure mode where the agent moves the camera away from the scene and the scripted policy stalls because the target is never detected as visible.

The `_update_rgb_buffer()` method renders both cameras each step: first the fixed camera (stored in `self._fixed_image`), then the pivoted camera (stored in `self._rgb_buffer` and returned as the observation).

3.5 Contact and Collision Modeling

SOFA's collision detection pipeline (LocalMinDistance, alarmDistance=5 mm, contactDistance=0.5 mm) is used with a Generic constraint solver. Seven separate ContactListener objects monitor the following pairs:

- Cauter–Gallbladder, Cauter–Liver, Gripper–Liver (base task collisions)
- Camera–Cauter, Camera–Gripper, Camera–Liver, Camera–Gallbladder (camera safety)

Camera collision geometry consists of 35 spheres of radius 1 mm placed along the camera shaft at 2 mm intervals from $z = -50$ to $z = +20$. This approximation matches the collision model approach used for the surgical instruments and provides conservative but reliable contact detection.

3.6 Task Structure and Phase Logic

The task is structured into three sequential phases:

1. **GRASP:** The Gripper must approach and grasp the gallbladder's infundibulum region. Phase active until a valid grasp (springs on both jaw and shaft in the graspable region) is established.
2. **LIFT:** With the gallbladder grasped, the Gripper must retract to expose the POI (as visible to the fixed overhead camera) while limiting gallbladder–liver overlap. Phase active until target is visible and overlap $\leq 10\%$.
3. **TOUCH:** The Cauter must approach and activate at the POI location (distance < 5 mm). Phase ends when the cauter is activated at the target.

A terminal **DONE** state is entered upon task success or on grasp loss (when `losing_grasp_ends_episode=True`). This is not a surgical phase but an episode termination condition.

Phase transitions are determined by the `_get_reward()` method in the base environment (`base_env.py`) at each step. The active phase is stored in `self.active_phase` and is directly available to the camera policy as an observation component.

3.7 Environment Reset and Randomization

On each `env.reset()` call:

- The POI position on the gallbladder is randomized (if enabled).
- Initial PTSD states for the Gripper and Cauter are perturbed by random noise: $\pm[10^\circ, 10^\circ, 45^\circ, 10\text{mm}]$ and jaw/activation noise respectively.
- Gallbladder surface tracking points are re-sampled using farthest-point sampling.
- The pivotized camera is reset to its default initial state.

Episode randomization ensures the learned policy generalizes across a distribution of initial configurations rather than overfitting to a single scene layout.

Chapter 4

Method

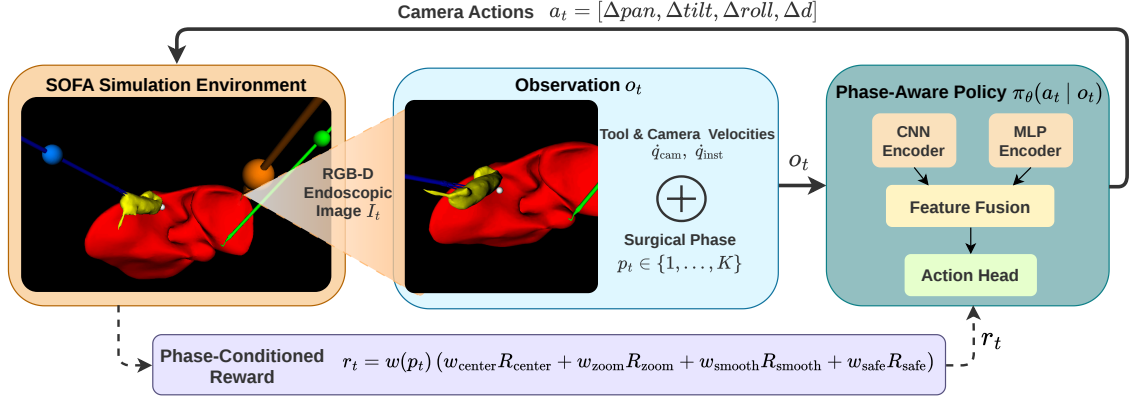


Figure 4.1: System Overview: The reinforcement learning agent receives visual observations, instrument and camera velocities, and phase information; outputs 4-DOF velocity commands to the pivoted camera; and receives a phase-conditioned reward.

4.1 Markov Decision Process Formulation

Autonomous camera control is formulated as a Markov Decision Process (MDP) $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, r, \gamma)$, where:

- $\mathcal{S}$: The state space, comprising visual observations of the surgical scene, instrument and camera velocity cues, and an explicit surgical phase encoding.
- $\mathcal{A}$: The action space, consisting of incremental 4-DOF velocity commands for the pivoted camera.
- $\mathcal{P}$: The transition dynamics, governed by the SOFA physics simulation and the RCM-constrained camera kinematic model.
- $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$: The phase-conditioned dense reward function detailed in Section 4.7.
- $\gamma \in (0, 1)$: Discount factor, set to $\gamma = 0.99$.

4.2 Observation Space

The policy receives a composite observation at each timestep:

$$o_t = \left(I_t, \mathbf{q}_t^{cam}, \mathbf{q}_t^{gripper}, \mathbf{q}_t^{cauter}, p_t \right) \quad (4.1)$$

where each component is defined as follows.

4.2.1 Visual Observation I_t

$I_t \in \mathbb{R}^{H \times W \times 4}$ is an RGB-D image rendered from the pivoted camera's current viewpoint. During training, $H = W = 256$ pixels (reduced for computational efficiency). During evaluation, $H = W = 1024$ pixels (resized to 256 during inference).

The depth channel is appended to the RGB image to provide structural information about instrument depth and proximity. No segmentation masks or annotated keypoints are provided - the policy must learn to infer instrument locations from raw pixel observations.

4.2.2 Velocity Observations $\dot{\mathbf{q}}$

Instrument velocities are computed as finite differences of PTSD states divided by the control timestep $\Delta t = 0.1$ s:

$$\dot{\mathbf{q}}_t^{cam} = \frac{\mathbf{q}_t^{cam} - \mathbf{q}_{t-1}^{cam}}{\Delta t} \in \mathbb{R}^4 \quad (4.2)$$

$$\dot{\mathbf{q}}_t^{gripper} = \frac{\mathbf{q}_t^{gripper} - \mathbf{q}_{t-1}^{gripper}}{\Delta t} \in \mathbb{R}^5 \quad (4.3)$$

$$\dot{\mathbf{q}}_t^{cauter} = \frac{\mathbf{q}_t^{cauter} - \mathbf{q}_{t-1}^{cauter}}{\Delta t} \in \mathbb{R}^5 \quad (4.4)$$

Velocity observations encode temporal dynamics without requiring absolute pose information. They allow the policy to distinguish stationary from moving instruments, anticipate fast motion, and regulate its own motion rate.

4.2.3 Phase Encoding p_t

$p_t \in \{0, 1, 2\}$ is a scalar integer encoding the current surgical phase:

$$p_t = 0 \quad (\text{GRASP})$$

$$p_t = 1 \quad (\text{LIFT})$$

$$p_t = 2 \quad (\text{TOUCH})$$

This signal is provided by the simulation environment’s internal phase state machine. In clinical deployment, it would be provided by a real-time phase recognition module (e.g., TeCNO [5]) or by a surgeon-operated foot pedal.

The phase signal is a scalar in the range $[0, 2]$ (matching the `ObservationType.RGBD_PTSD_VELOCITY_PHASE` space) and is concatenated with the velocity vectors.

4.3 Action Space

The action space is continuous and four-dimensional:

$$a_t = \left(\Delta \dot{\theta}_t^{pan}, \Delta \dot{\theta}_t^{tilt}, \Delta \dot{\theta}_t^{roll}, \Delta d_t \right) \in [-1, 1]^4 \quad (4.5)$$

Actions are normalized to $[-1, 1]$ and scaled to physical velocity ranges by:

$$\mathbf{u}_t = \Delta t \cdot \mathbf{v}_{max} \odot a_t \quad (4.6)$$

where $\mathbf{v}_{max} = [15^\circ/\text{s}, 15^\circ/\text{s}, 25^\circ/\text{s}, 25 \text{ mm/s}]$ (the first four components of the instrument maximum velocity array). An action of $a_t = 1.0$ in the pan dimension therefore commands a pan velocity of $15^\circ/\text{s}$ for 0.1 s, resulting in a 1.5° pan change per step.

4.4 Remote Center of Motion Constraint

All camera commands are executed within the RCM-constrained kinematic model provided by the SOFA `PivotizedCamera` template. The RCM constraint enforces that the camera shaft always passes through the fixed trocar insertion point $\mathbf{p}_{rcm} = [-155, -180, -260] \text{ mm}$. Physically, this means that only rotations about the trocar point and translations along the shaft are permissible - the camera cannot translate in the workspace except through the combined effect of rotation and depth change. This constraint is guaranteed by construction in the PTSD parameterization and does not require explicit enforcement in the reward function.

4.5 Scripted Tool Policy

In the camera-only environment (`camera_env.py`), the Gripper and Cauter are controlled by a scripted finite-state machine that executes the task autonomously. This allows the RL agent to focus solely on camera placement optimization without having to learn instrument control simultaneously.

The scripted policy operates as follows:

Gripper FSM The state machine transitions through: *approach* (move PTSD toward the optimal grasp state) $\rightarrow$ *close* (reduce jaw angle to 5°) $\rightarrow$ *settle* (wait 10 steps for grasp springs to engage) $\rightarrow$ *lift* (gradually reduce depth to expose the POI). Up to five reattempts are made before falling back to noise-free optimal state.

Cauter IK Once the target is visible (detected via the fixed camera), the cauter is driven by inverse kinematics to the target. The function `target_pose_to_ptsd()` maps the target world position to a PTSD state using the RCM pose and a tool tip offset of 14.4 mm.

Human Noise Model When `human_noise=True`, each scripted tool motion is perturbed by a composite noise model described in Section 4.6. This prevents the camera policy from overfitting to perfectly smooth tool trajectories.

4.6 Stochastic Instrument Motion Model

To prevent policy overfitting to deterministic scripted trajectories, a composite noise model perturbs each scripted instrument command at every control timestep. The noisy target is computed as:

$$\tilde{\mathbf{x}}^* = \mathbf{x}^* + s(d_t) \cdot (\mathbf{d}_t + \boldsymbol{\varepsilon}_t) \quad (4.7)$$

where $\mathbf{d}_t$ is a correlated low-frequency drift, $\boldsymbol{\varepsilon}_t$ is independent high-frequency tremor, and $s(d_t)$ is a distance-dependent attenuation factor.

Correlated Drift (AR(1)) Models sustained hand deviations:

$$\mathbf{d}_t = \rho \mathbf{d}_{t-1} + \boldsymbol{\eta}_t, \quad \boldsymbol{\eta}_t \sim \mathcal{N}(\mathbf{0}, \sigma_{drift}^2 \mathbf{I}) \quad (4.8)$$

with $\rho = 0.98$, $\sigma_{drift} = 3.0$ mm. The high correlation coefficient produces smooth, slowly-evolving deviations that mimic sustained arm movement.

High-Frequency Tremor Independent Gaussian noise models physiological tremor:

$$\boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \sigma_{tremor}^2 \mathbf{I}), \quad \sigma_{tremor} = 0.5 \text{ mm} \quad (4.9)$$

Distance-Dependent Scaling Noise is attenuated near task-completion targets:

$$s(d_t) = \begin{cases} 0 & \text{if } d_t < d_{dead} = 10 \text{ mm} \\ \min(d_t/d_{scale}, 1) & \text{otherwise} \end{cases} \quad (4.10)$$

where $d_{scale} = 60$ mm defines the full-noise threshold. This simulates a surgeon applying higher precision near the target.

The jaw angle and cauter activation dimension are never perturbed (noise applied only to the first four PTSD dimensions). This prevents physiologically unrealistic jaw flutter.

4.7 Phase-Conditioned Reward Design

Unlike standard manipulation tasks where success can be defined by a simple binary signal (e.g., object grasped or dropped), visual quality is a continuous, subjective metric. A binary 1/0 reward for camera control is ill-posed and provides an insufficient gradient for learning smooth, cinematic framing. Therefore, a core contribution of this work is a non-trivial, dense reward formulation that explicitly quantifies cinematic rules.

The total reward at each timestep is a weighted sum of four components:

$$R_{total} = \omega_c R_{center} + \omega_z R_{zoom} + \omega_j R_{jitter} + \omega_{coll} R_{coll} \quad (4.11)$$

with $\omega_c = 1.0$, $\omega_z = 2.5$, $\omega_j = 2.0$, $\omega_{coll} = 2.0$ (experimentally chosen).

4.7.1 Instrument Tip Point Projection

Before computing any reward term, the 3D positions of relevant scene objects are projected into the pivoted camera's 2D image plane. For each instrument (Gripper, Cauter), five points are sampled along the tip region (0 to 30 mm up the shaft from the tip toward the RCM) and projected using the camera's intrinsic and extrinsic matrices:

$$(r_i, c_i) = \pi \left(\mathbf{p}^{tip} + \frac{i}{N-1} \cdot L_{tip} \cdot \hat{\mathbf{u}}_{shaft}, \mathbf{K}_{cam}, \mathbf{T}_{cam} \right) \quad (4.12)$$

where $\pi(\cdot)$ is the perspective projection function, $\hat{\mathbf{u}}_{shaft}$ is the unit vector from tip toward RCM, $L_{tip} = 30$ mm, and $N = 5$.

The POI is projected as a single point at its 3D world position.

4.7.2 Phase-Dependent Attention Weights

Phase-dependent weights w_k for objects $k \in \{\text{Gripper, Cauter, POI}\}$ are defined as:

Phase	w_{grip}	w_{caut}	w_{poi}
GRASP	0.8	0.2	0.0
LIFT	0.4	0.0	0.6
TOUCH	0.2	0.4	0.4

During GRASP phase, all attention is directed to the Gripper (the active instrument). During LIFT, attention shifts to the POI (whose visibility must be confirmed). During TOUCH, the Cauter and POI share priority while the Gripper remains in peripheral view.

4.7.3 Centering Score and Visibility

For each object, a centering score $S_k \in (-\infty, 1]$ is computed from its projected pixel coordinates:

$$S_k = \text{mean}_i \left[\text{clip} \left(1 - \sqrt{y_i^2 + x_i^2}, -\infty, 1.0 \right) \right] \quad (4.13)$$

where $y_i = (r_i - H/2)/(H/2)$, $x_i = (c_i - W/2)/(W/2)$ are normalized pixel coordinates centered at the image center. A deadzone plateau clamps scores above 0.9 to exactly 1.0, preventing micro-jitter near the image center.

Visibility fraction V_k is the fraction of sampled tip points within the image bounds.

The phase-conditioned combined centering reward is:

$$R_{center} = \left(\sum_k w_k S_k \right) \cdot \left(1 + \lambda_{vis} \sum_k w_k V_k \right) \quad (4.14)$$

where $\lambda_{vis} = 4.0$. The multiplicative visibility boost ensures that high centering scores are only rewarded when instruments are actually in frame. An instrument that is off-frame contributes a negative centering score, providing a gradient that pulls the camera back even when visibility is zero.

4.7.4 Zoom Reward

The zoom reward penalizes deviations from a target visual spread of instruments in the image. The spread is measured as the maximum pixel distance of any projected point from the centroid of all relevant projected points, normalized by $\min(H, W)/2$:

$$\sigma = \frac{\max_i \|\mathbf{p}_i - \bar{\mathbf{p}}\|}{\min(H, W)/2} \quad (4.15)$$

$$R_{zoom} = \exp\left(-\frac{1}{2}\left(\frac{\sigma - \sigma_{target}}{\sigma_{width}}\right)^2\right) \quad (4.16)$$

with $\sigma_{target} = 0.9$ and $\sigma_{width} = 0.15$. This Gaussian reward is maximized when instruments span approximately 90% of the image half-width, providing both contextual depth and instrument detail simultaneously.

4.7.5 Motion Jitter Penalty

$$R_{jitter} = -(\alpha \|\dot{\mathbf{q}}_t^{cam}\| + \beta \|\ddot{\mathbf{q}}_t^{cam}\|) \quad (4.17)$$

where $\alpha = 0.02$, $\beta = 0.001$, and the camera acceleration $\ddot{\mathbf{q}}_t^{cam} = (\dot{\mathbf{q}}_t^{cam} - \dot{\mathbf{q}}_{t-1}^{cam})/\Delta t$. This penalizes both camera speed and jerk, encouraging gradual, smooth motion.

4.7.6 Collision Penalty

$$R_{coll} = \begin{cases} -1 & \text{if any camera contact detected} \\ 0 & \text{otherwise} \end{cases} \quad (4.18)$$

The binary formulation penalizes all collisions equally, regardless of contact count. This avoids instabilities from large proportional penalties and encourages the agent to maintain a comfortable clearance from instruments and anatomy.

4.8 Policy Architecture and Training

The Phase-Aware policy is trained using Proximal Policy Optimization (PPO) [18] implemented in Stable Baselines3.

Network Architecture The policy uses the `MultiInputPolicy` architecture. Visual observations (RGB-D, 256×256) are processed by the `NatureCNN` encoder: three convolutional layers with (32 filters, 8×8 , stride 4), (64 filters, 4×4 , stride 2), (64 filters, 3×3 , stride 1), producing a 256-dimensional visual feature vector. This is concatenated with the flattened velocity and phase vectors and passed through a two-layer MLP with 64 units per layer and $\tanh$ activations to produce both the actor (policy) and critic (value function) outputs.

PPO Hyperparameters Learning rate: 3×10^{-4} ; rollout length: 512 steps per environment; minibatch size: 64; optimization epochs per rollout: 10; GAE $\lambda = 0.95$; clip range: 0.2; total training steps: 5×10^6 .

Vectorized Training Four parallel SubprocVecEnv environments are used for data collection (total of $4 \times 512 = 2048$ transitions per PPO update). Episodes are limited to 250 steps per TimeLimit wrapper.

Chapter 5

Implementation

5.1 Code Architecture

The codebase is organized as follows:

```
Phase_Aware_ACC/  
├── phase_aware_env/  
│   ├── __init__.py -- Package initialiser  
│   ├── base_env.py -- Base GraspLiftTouchEnv  
│   ├── camera_env.py -- Camera-only env  
│   ├── scene_description.py -- SOFA scene graph  
│   ├── weighted_poi_heuristic_controller.py  
│   └── sofa_objects/ -- SOFA wrappers  
├── scripts/  
│   ├── train.py -- PPO training entry point  
│   ├── test.py -- Visual testing / rendering  
│   ├── evaluate.py -- RL policy quantitative evaluation  
│   └── evaluate_heuristic.py -- Heuristic baseline evaluation  
├── assets/ -- Instrument meshes & organ models  
├── sofa_env/ -- Vendored sofa_env library  
└── setup.py -- Package installation & dependencies
```

Figure 5.1: Project repository layout for reproducibility.

5.2 Base Environment: `base_env.py`

`GraspLiftTouchEnv` extends `SofaEnv` from the `sofa_env` library. It implements:

- **ObservationType:** Supports RGB, RGBD, and multiple PTSD-velocity combinations. The `RGBD_PTSD_VELOCITY_PHASE` type is used.
- **ActionType:** Supports `CONTINUOUS`, `POSITION`, and `DISCRETE`. Training uses `CONTINUOUS`.
- **Phase state machine:** Transitions determined in `_get_reward()` from reward features computed by `_get_reward_features()`.
- **Reward features:** A comprehensive dictionary including distances, velocities, grasping state, collision counts, target visibility, and overlap metrics, computed at each step from SOFA simulation state.

- **Episode info logging:** All reward components are accumulated into `episode_info` using shortened keys for TensorBoard.

5.3 Camera Environment: `camera_env.py`

`PivotizedCameraOnlyEnv1` extends `GraspLiftTouchEnv`:

- **Overrides** `_do_action` with `_do_action_camera_only`, which applies the camera action and then calls `_scripted_tools_step()`.
- **Overrides** `_update_rgb_buffer` with dual-camera rendering logic.
- **Overrides** `_get_reward` to return only the camera visibility reward (base task reward weights are zeroed out in the parent's `reward_amount_dict`).
- **Overrides** `_get_reward_features` to pass `self._fixed_image` to the base class (ensuring scripted tool logic uses the fixed camera) while computing camera reward features from the pivotized camera.
- Adds the `_calculate_extra_reward_features()` method implementing the full four-component camera reward.

Dual-camera rendering loop The rendering sequence at each step is:

1. Switch rendering target to fixed camera → render → store as `self._fixed_image`.
2. Switch rendering target to pivotized camera → render → store in `self._rgb_buffer`.
3. If `show_fixed_camera=True`, display fixed camera in the window (for debugging).

5.4 Heuristic Baseline

The analytic heuristic controller (`WeightedPOIHeuristicController`) computes a camera action in three steps:

1. **Weighted target:** Compute a phase-dependent weighted centroid in world space from the 3D positions of the Gripper tip, Cauter tip, and POI.
2. **Ideal PTSD:** Call `target_pose_to_ptsd()` to compute the ideal camera PTSD state that centers the weighted world point.
3. **Action scaling:** Compute the state delta $\Delta \mathbf{q} = \mathbf{q}^{ideal} - \mathbf{q}^{current}$, normalize by the maximum step size, and scale by a proportional gain based on the normalized pixel error magnitude. A deadband of 0.01 suppresses actions near the center.

Additionally, a depth regulation loop adjusts insertion depth based on the zoom score:

$$d^{target} = d^{current} + k_{zoom} \cdot (z^{target} - z^{current}) \cdot 10\text{mm} \quad (5.1)$$

with $k_{zoom} = 0.35$.

Optional Gaussian noise ($\sigma = 0.05$) can be added to simulate imperfect analytic control.

5.5 Training Pipeline: `train.py`

The training script supports:

- `-phased`: Enable phase-aware observation (adds p_t to observation).
- `-human-noise`: Enable stochastic instrument noise.
- `-frame-stack`: Stack multiple frames for temporal context (not used in primary experiments).
- `-load-model`: Resume training from an existing checkpoint.

TensorBoard logging tracks per-episode reward breakdowns via the `RewardComponent Callback`, which extracts reward components from the info dictionary at episode boundaries. The `EvalCallback` saves the best model based on mean evaluation episode reward over 10 evaluation episodes every 10,000 training steps.

5.6 Evaluation Pipeline

5.6.1 `evaluate.py` - RL Policy Evaluation

Loads a trained PPO model and runs deterministic rollouts over a configurable number of episodes. Computes per-phase aggregates for Visibility, Centering, Zoom, and Smoothness, and generates an IEEE-formatted publication-quality plot (`.png` + `.pdf`). Results are saved as `.txt`, `.csv`, and `.npz` (for cross-method comparison plots).

5.6.2 `evaluate_heuristic.py` - Heuristic Evaluation

Runs the `WeightedPOIHeuristicController` over the same number of episodes with identical environment settings. Computes all metrics in the same format as `evaluate.py` for direct numerical comparison. Additional metrics specific to the heuristic include oscillation rate (frequency of action sign changes), phase settle steps (steps to achieve ≥ 0.7 visibility after a phase transition), and mean bootstrap steps.

Chapter 6

Results and Analysis

6.1 Evaluated Controllers

We evaluate and compare the performance of three camera controllers:

- **Phase-Aware RL (Proposed):** The primary learned policy conditioned explicitly on the surgical phase signal p_t .
- **Phase-Blind RL (Ablation):** An ablation model that removes the phase encoding from the observation space while retaining identical neural architecture, reward function, and training protocol. This isolates the contribution of explicit phase conditioning at inference time.
- **Weighted POI Heuristic (Baseline):** A reactive analytic controller that computes a phase-dependent weighted target from relevant tool keypoints, maps it to an RCM-consistent camera configuration using inverse pivot kinematics, and generates velocity commands proportional to the normalized pixel error. Depth is regulated using a zoom-error feedback term. This baseline represents classical visual servo-inspired control without learning.

6.2 Evaluation Setup

All three controllers are evaluated over 50 episodes using stochastic instrument motion (`human_noise=True`) and a maximum of 500 steps per episode. A fixed base seed is used with episode-indexed increments to ensure reproducibility across methods. Evaluation uses the 1024×1024 rendering resolution (downsampled to 256×256 for policy inference).

6.3 Aggregate Performance

Table 6.1 summarizes the aggregate performance metrics averaged across all phases and episodes.

Table 6.1: Quantitative camera control performance (mean over 50 episodes). $\uparrow$ higher is better; $\downarrow$ lower is better.

Controller	Vis. $\uparrow$	Center $\uparrow$	Zoom $\uparrow$	Jitter $\downarrow$
Weighted POI Heuristic	0.86	0.42	0.11	0.56
Phase-Blind RL	0.90	0.39	0.21	1.02
Phase-Aware RL	0.99	0.59	0.19	0.34

Phase-Aware RL achieves the highest visibility (0.99), highest centering (0.59), and lowest jitter (0.34) of all three controllers. Relative to the oracle heuristic, this represents a 40% improvement in centering (0.59 vs. 0.42) and a 39% reduction in jitter (0.34 vs. 0.56). The mean episode return is also highest at 600.61.

Heuristic achieves moderate centering (0.42) and visibility (0.86) but low zoom (0.11) and moderate jitter (0.56). Despite operating on ground-truth 3D positions, the purely reactive proportional control architecture cannot balance the zoom-centering trade-off under stochastic instrument motion.

Phase-Blind RL achieves good visibility (0.90) but the lowest centering (0.39) and highest jitter (1.02). The absence of the phase signal at inference time causes the policy to fail at adapting its tracking priority as the task evolves, resulting in high variance and erratic motion.

6.4 Phase-Wise Centering Analysis

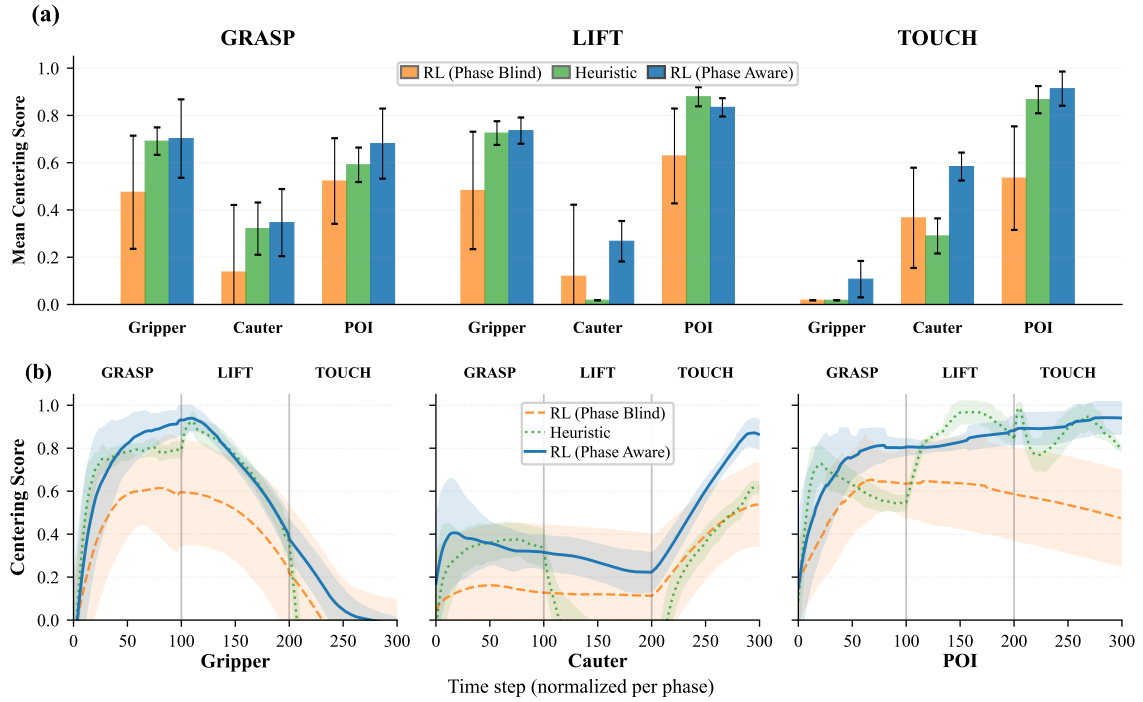


Figure 6.1: Comparative analysis of camera control performance across the three surgical phases. (a) Grouped bar chart of mean centering scores for Gripper, Cauter, and POI per phase. Phase-Aware RL (blue) consistently achieves the highest accuracy by dynamically adapting tracking priorities. (b) Temporal traces of centering scores with ± 1 std shading over 50 episodes. Vertical dashed lines indicate phase transitions.

Figure 6.1 and the phase-wise breakdown reveal several important observations:

GRASP phase Both the heuristic (grripper centering: 0.691) and Phase-Aware RL (0.702) achieve similar high gripper centering. With the cauter weight at 0.2 and POI weight at 0.0, single-target tracking is relatively simple. The Phase-Blind model (0.475) performs significantly worse, suggesting difficulty in determining which instrument to prioritize.

LIFT phase The heuristic achieves the highest POI centering in this phase (0.879), driven by the direct inverse kinematics that points the camera exactly at the phase-weighted centroid of Gripper and POI. The Phase-Aware RL achieves 0.834. However, the heuristic’s strict weight assignment (cauter weight = 0.0) causes the cauter to frequently drift off-frame. This depresses the heuristic’s *mean* centering even in phases where its primary target centering is excellent.

TOUCH phase The Phase-Aware RL achieves the highest cauter centering (0.584) among all controllers. The Cauter and POI must be jointly maintained in frame during

fine dissection - a multi-target task that the reactive heuristic handles poorly due to its centroid-only approach.

Mean vs. Peak The heuristic’s aggregate centering (0.42) is significantly lower than Phase-Aware RL (0.59) despite having perfect geometric knowledge. This is a key finding: the advantage of learned control over analytic control is not in any single phase, but in the globally balanced multi-target tracking that the heuristic cannot achieve by construction.

6.5 Temporal Stability

The temporal traces in Figure 6.1(b) reveal important differences in stability:

- The **heuristic** exhibits small but consistent oscillations throughout all phases. Because it recomputes the ideal centroid from ground-truth state at every timestep, it directly amplifies the high-frequency tremor injected by the noise model. This corresponds to cumulative jitter of 0.56.
- The **Phase-Aware RL** policy maintains stable traces with low variance (± 0.05), matching the consistency of the deterministic heuristic despite operating under noise. The implicit temporal filtering learned from the jitter penalty term means the policy maintains a stable viewing region and only reorients for sustained motion trends.
- The **Phase-Blind RL** exhibits high variance (± 0.11) particularly around phase transitions, where without a phase signal the policy fails to adapt its tracking priority cleanly. This is the dominant source of its elevated jitter (1.02).

6.6 Statistical Significance

A two-sample t -test on per-episode POI centering scores during the TOUCH phase confirms that the Phase-Aware RL advantage is statistically significant:

- Phase-Aware RL: 0.913 ± 0.066
- Phase-Blind RL: 0.535 ± 0.218
- $p < 0.001$

6.7 Episode Return and Outcomes

Mean episode return: Phase-Aware RL (600.61) > Phase-Blind RL (421.71) > Heuristic (380.89). The higher return of Phase-Aware RL reflects both better centering reward and lower jitter/collision penalties. The heuristic’s lower return compared to Phase-Blind RL

results primarily from its poor zoom regulation (0.11), which accumulates a substantial negative reward contribution.

6.8 Zoom Regulation Analysis

All three controllers achieve moderate zoom scores (0.11–0.21). The heuristic achieves the lowest (0.11): because depth adjustments are computed from geometric centroid changes, stochastic instrument clustering during noisy motion causes repeated depth overcorrection. The RL policies adopt more conservative depth strategies that prioritize global stability over exact spread matching, resulting in slightly higher zoom scores (0.19–0.21). The narrow Gaussian zoom reward inherently conflicts with centering and jitter minimization — achieving the Gaussian peak requires large depth adjustments that incur jitter penalties.

Chapter 7

Discussion and Limitations

7.1 Key Findings Summary

The results establish three principal findings:

1. **Phase conditioning improves multi-target centering.** The Phase-Aware RL policy achieves significantly higher mean centering than both the phase-blind ablation and the oracle geometric heuristic. This improvement is most pronounced in multi-target phases (LIFT, TOUCH) where the heuristic’s strict weight assignments cause peripheral instruments to drift off-frame.
2. **Learned temporal filtering reduces jitter.** The RL policy implicitly learns to suppress high-frequency instrument tremor by maintaining a stable viewing region. This effect cannot be replicated by the reactive heuristic without manual temporal filtering, which would introduce lag at phase transitions.
3. **Privileged geometric access does not guarantee superior performance.** Despite operating on ground-truth 3D instrument positions, the analytic heuristic is outperformed by the learned policy in centering, jitter, and episode return. This demonstrates that the combination of procedural context and learned temporal dynamics provides advantages that pure geometric control cannot achieve.

7.2 Ablation: Contribution of Phase Signal

The comparison between Phase-Aware RL and Phase-Blind RL isolates the contribution of the explicit phase signal. Both models share identical architecture, reward function, and training protocol - the only difference is whether p_t is included in the observation. The significant performance gap (centering: 0.59 vs. 0.39; jitter: 0.34 vs. 1.02; return: 600.61 vs. 421.71) demonstrates that the phase signal is not merely decorative - it provides essential context for adapting tracking priorities at inference time.

It is worth noting that both models are trained with the *same* phase-conditioned reward. The Phase-Blind model therefore experiences the correct reward shaping during training but cannot use the phase signal to modulate its behavior. This ablation specifically tests whether the policy benefits from *knowing* the current phase, independent of the reward structure.

7.3 Comparison with Prior Work

The heuristic baseline in this work is more sophisticated than the visual servo baselines commonly reported in the literature: it uses exact inverse pivot kinematics (rather than simple proportional pixel error), phase-dependent weighted centroids (rather than a fixed tool priority), and zoom-error depth regulation. Despite this, it is outperformed by the learned policy. This result suggests that even analytically strong reactive controllers are fundamentally limited by their inability to learn temporal filtering and multi-objective trade-off balancing.

7.4 Simulation-Only Evaluation

All experiments are conducted exclusively in SOFA simulation. The simulation captures the kinematics, collision behavior, and rigid/deformable dynamics of the scene with reasonable fidelity, but does not model:

- Real-world perception noise (camera sensor noise, motion blur, lens distortion)
- Actuation delays and backlash in physical robotic systems
- Lighting variability, blood/fluid on the lens
- Full tissue deformation dynamics beyond FEM approximations
- Inter-patient anatomical variability

Physical validation on a robotic test bench (e.g., with a da Vinci surgical system or a laparoscopic training setup) is essential before clinical translation. The current results demonstrate feasibility and provide a principled framework for future hardware experiments.

7.5 Oracle Phase Signal

The framework assumes clean, synchronous access to the surgical phase signal. In a real surgical setting, the phase must be inferred from video or other sensors - a process that introduces latency, misclassification errors, and ambiguous transition periods. Future work should evaluate robustness to:

- Phase recognition latency (evaluating with delayed phase signals)
- Phase misclassification (adding noise to the phase input during inference)
- Finer-grained phase decomposition (sub-phases within each current phase)

7.6 Limitations of the Reward Design

The zoom reward uses a fixed target spread parameter ($\sigma_{target} = 0.9$) that may not be appropriate for all procedural contexts. During rapid Gripper retraction in the LIFT phase, the optimal camera depth changes quickly, but the zoom reward does not adapt to this. A phase-conditioned zoom target (using different σ_{target} per phase) may improve zoom regulation.

The centering reward’s deadzone plateau (score clamped to 1.0 for instruments within 10% of center) prevents fine centering precision. A more graduated reward that rewards continued improvement even within the deadzone might yield better centering accuracy at the cost of increased jitter.

7.7 Extensions and Future Work

- **End-to-end joint training:** Train the camera policy simultaneously with the instrument controllers rather than using a scripted policy, enabling co-optimization of visualization and task execution.
- **Noisy phase input robustness:** Evaluate and train with noisy or delayed phase signals to assess and improve tolerance to real-world phase recognition imperfections.
- **Real-world deployment:** Deploy the trained policy on a physical robot (e.g., a da Vinci Research Kit) with appropriate sim-to-real transfer techniques (domain randomization, fine-tuning on physical data).
- **Human-in-the-loop evaluation:** Evaluate visual comfort and task performance in a user study with surgeons, measuring metrics such as visual fatigue, scene understanding, and procedural efficiency.
- **Finer phase granularity:** Extend to finer-grained phase decompositions (e.g., sub-phases of the cholecystectomy workflow) to capture more nuanced visualization requirements.
- **Temporal observation:** Evaluate the benefit of frame stacking or recurrent policies (LSTM/GRU) for providing richer temporal context beyond the single-step velocity observation.

Chapter 8

Conclusion

This project presents a complete design, implementation, and evaluation of a phase-aware reinforcement learning framework for autonomous laparoscopic camera control. A central technical contribution of this framework is the formulation of a dense, multi-objective reward signal that mathematically operationalizes cinematic visual quality. By moving beyond simple binary task-completion metrics, the policy is guided to learn a nuanced balance between centering, zoom, and stability.

Furthermore, the core insight driving this approach is straightforward but impactful: visualization requirements are not static. Conditioning the camera control policy on the current surgical phase enables it to dynamically adapt these framing objectives as the procedure evolves.

The implementation spans a custom physics-based SOFA simulation environment with deformable tissue, a dual-camera rendering architecture, a stochastic human-like noise model for scripted instrument control, and a complete training and evaluation pipeline using PPO via Stable Baselines3.

Evaluation against both a phase-blind RL ablation and an oracle geometric heuristic controller demonstrates that:

- Phase conditioning yields superior multi-target centering that neither baseline can achieve.
- Learned temporal filtering produces smoother camera motion than reactive proportional control.
- Privileged geometric access (as in the heuristic) does not compensate for the absence of learned temporal dynamics and multi-objective balancing.

These results have been submitted as a full research paper to the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) 2026. The codebase has been archived at <https://zenodo.org/records/18830024> for reproducibility.

The broader implication of this work is that workflow-level context - surgical phase being the simplest form - is a powerful and underutilized signal for autonomous surgical assistance. Translating this principle to physical robotic systems and extending it to

finer procedural granularity represents an important direction for making robotic-assisted surgery more intelligent, safe, and ergonomic.

Appendix A

Network Architecture and Hyperparameters

Table A.1: PPO training hyperparameters

Parameter	Value
Learning rate	3×10^{-4}
Rollout length (n_{steps})	512
Minibatch size	64
Optimization epochs	10
Discount (γ)	0.99
GAE (λ)	0.95
Clip range	0.2
Total timesteps	5×10^6
Parallel environments	4 (SubprocVecEnv)
Episode limit	250 steps (TimeLimit)
Device	CUDA GPU

Table A.2: CNN encoder architecture (NatureCNN)

Layer	Filters	Kernel	Stride	Output
Conv1	32	8×8	4	$31 \times 31 \times 32$
Conv2	64	4×4	2	$14 \times 14 \times 64$
Conv3	64	3×3	1	$12 \times 12 \times 64$
Flatten	—	—	—	9216
Linear	256	—	—	256

The 256-dimensional visual feature vector is concatenated with the $4 + 5 + 5 + 1 = 15$ -dimensional non-visual observation vector (camera velocity, gripper velocity, cauter velocity, phase scalar) to form a 271-dimensional input to the MLP head (two layers of 64 units, tanh activations).

Appendix B

Reward Component Weights

Table B.1: Reward function parameters

Parameter	Symbol	Value
Centering weight	ω_c	1.0
Zoom weight	ω_z	2.5
Jitter weight	ω_j	2.0
Collision weight	ω_{coll}	2.0
Visibility boost base	–	1.0
Visibility boost multiplier	λ_{vis}	4.0
Zoom target spread	σ_{target}	0.9
Zoom spread tolerance	σ_{width}	0.15
Jitter velocity coefficient	α	0.02
Jitter acceleration coefficient	β	0.001
Tip sampling length	L_{tip}	30.0 mm
Tip sampling points	N	5
Centering deadzone	–	0.9 (clamped to 1.0)

Appendix C

Stochastic Noise Model Parameters

Table C.1: Stochastic instrument motion noise parameters

Parameter	Description	Value
ρ (drift correlation)	AR(1) process coefficient	0.98
σ_{drift}	Drift magnitude	3.0 mm
σ_{tremor}	Tremor magnitude	0.5 mm
d_{dead}	Noise-free zone radius	10.0 mm
d_{scale}	Full-noise threshold distance	60.0 mm

Appendix D

Simulation Constants and Scene Parameters

Table D.1: Simulation and scene parameters

Parameter	Value
Control timestep Δt	0.1 s
SOFA physics dt	0.1 s
Gravity	(0, 9.81, 0) m/s ²
Constraint solver	GenericConstraintSolver (maxIt=1000, tol= 10^{-3})
Collision detection	LocalMinDistance (alarm=5 mm, contact=0.5 mm)
Camera vertical FOV	45°
Camera z-near / z-far	20 mm / 1500 mm
Training image resolution	256 × 256
Evaluation image resolution	1024 × 1024
Max pan/tilt velocity	15 °/s
Max roll velocity	25 °/s
Max depth velocity	25 mm/s
Goal tolerance (cauter-POI)	5 mm
Max gallbladder-liver overlap	10%
Episode step limit	250 (train) / 500 (eval)

Appendix E

Worked Reward Calculation Example

This appendix provides a complete numerical trace of the reward calculation for a representative timestep in the LIFT phase (Weights: $w_{grip} = 0.4, w_{caut} = 0.0, w_{poi} = 0.6$). All values are drawn from a typical mid-episode step during evaluation.

Projected observations

- Gripper tip: visible, near center. $S_{grip} = 0.75, V_{grip} = 1.0$.
- Cauter tip: partially visible. $S_{caut} = 0.40, V_{caut} = 0.6$.
- POI: visible, slightly off-center. $S_{poi} = 0.55, V_{poi} = 1.0$.

Centering reward

$$\begin{aligned}\text{total_vis} &= 0.4(1.0) + 0.0(0.6) + 0.6(1.0) = 1.0 \\ \text{total_centering} &= 0.4(0.75) + 0.0(0.40) + 0.6(0.55) = 0.63 \\ \text{vis_boost} &= 1.0 + 4.0 \cdot 1.0 = 5.0 \\ R_{center} &= 0.63 \times 5.0 \times 1.0 = 3.15\end{aligned}$$

Zoom reward Instrument spread: $\sigma = 0.85$ (slightly under target 0.9).

$$\begin{aligned}R_{zoom} &= \exp\left(-\frac{1}{2}\left(\frac{0.85 - 0.9}{0.15}\right)^2\right) = e^{-0.0556} \approx 0.946 \\ \omega_z R_{zoom} &= 0.946 \times 2.5 = 2.365\end{aligned}$$

Jitter penalty Camera moved 2° Pan this step: $\dot{\mathbf{q}}^{cam} = [20, 0, 0, 0]^\circ/\text{s}$. Similar motion last step: $\ddot{\mathbf{q}}^{cam} \approx [0, 0, 0, 0]$.

$$\begin{aligned}R_{jitter} &= -(20 \times 0.02 + 0 \times 0.001) = -0.40 \\ \omega_j R_{jitter} &= -0.40 \times 2.0 = -0.80\end{aligned}$$

Collision penalty No contacts: $R_{coll} = 0.0$.

Total reward $R_{total} = 3.15 + 2.365 - 0.80 + 0.0 = \mathbf{4.715}$

Bibliography

- [1] C. Gruijthuijsen, L. C. Garcia-Peraza-Herrera, G. Borghesan, D. Reynaerts, J. Deprest, S. Ourselin, T. Vercauteren, and E. Vander Poorten, “Robotic endoscope control via autonomous instrument tracking,” *Frontiers in Robotics and AI*, vol. 9, p. 832208, 2022.
- [2] C. Zhang, W. Zhu, J. Peng, Y. Han, and W. Liu, “Visual servo control of endoscope-holding robot based on multi-objective optimization: System modeling and instrument tracking,” *Measurement*, vol. 211, p. 112658, 2023.
- [3] S. W. Wong and P. Crowe, “Visualisation ergonomics and robotic surgery,” *Journal of Robotic Surgery*, vol. 17, pp. 1873–1878, 2023.
- [4] T. Da Col, G. Caccianiga, M. Catellani, A. Mariani, M. Ferro, G. Cordima, E. De Momi, G. Ferrigno, and O. de Cobelli, “Automating endoscope motion in robotic surgery: A usability study on da vinci-assisted ex vivo neobladder reconstruction,” *Frontiers in Robotics and AI*, vol. 8, p. 707704, 2021.
- [5] T. Czempiel, M. Paschali, M. Keicher, W. Simson, H. Feussner, S. T. Kim, and N. Navab, “Tecno: Surgical phase recognition with multi-stage temporal convolutional networks,” *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pp. 343–352, 2020.
- [6] X. Ding, X. Yang, T. Wendler, and N. Navab, “Less is more: Surgical phase recognition from timestamp supervision,” *IEEE Transactions on Medical Imaging*, vol. 42, no. 7, pp. 2023–2034, 2023.
- [7] K. Shinozuka, S. Turuda, A. Fujinaga, H. Nakanuma, H. Kawamura, Y. Matsunobu, Y. Tanaka, T. Kamiyama, K. Ebe, Y. Endo, T. Etoh, M. Inomata, and T. Tokuyasu, “Artificial intelligence software available for medical devices: surgical phase recognition in laparoscopic cholecystectomy,” *Surgical Endoscopy*, vol. 36, pp. 7444–7452, 2022.
- [8] M. Wagner, A. Bihlmaier, H. G. Kenngott, P. Mietkowski, P. M. Scheikl, S. Bodendstedt, A. Schiepe-Tiska, J. Vetter, F. Nickel, S. Speidel *et al.*, “A learning robot for cognitive camera control in minimally invasive surgery,” *Surgical Endoscopy*, vol. 35, pp. 5365–5374, 2021.
- [9] S. Eslamian, S. Reisner, and A. Pandya, “Development and evaluation of an autonomous camera control algorithm on the da vinci surgical system,” *The Interna-*

- tional Journal of Medical Robotics and Computer Assisted Surgery*, vol. 16, no. 2, p. e2079, 2020.
- [10] E. Fazzari and C. Stefanini, “Deep reinforcement learning for surgical robotics with state and image information: A survey,” *Research Square (Preprint)*, 2026.
- [11] Y. Ou, S. Zargarzadeh, and M. Tavakoli, “Robot learning incorporating human interventions in the real world for autonomous surgical endoscopic camera control,” *Journal of Medical Robotics Research*, 2023.
- [12] A. T. Bourdillon, A. Garg, H. Wang, Y. J. Woo, M. Pavone, and J. Boyd, “Integration of reinforcement learning in a virtual robotic surgical simulation,” *Surgical Innovation*, vol. 30, no. 1, pp. 94–102, 2023.
- [13] P. M. Scheikl, B. Gyenes, R. Younis, C. Haas, G. Neumann, M. Wagner, and F. Mathis-Ullrich, “Lapgyim-an open source framework for reinforcement learning in robot-assisted laparoscopic surgery,” *Journal of Machine Learning Research*, vol. 24, no. 368, pp. 1–42, 2023.
- [14] T. Nageli, L. Meier, A. Domahidi, J. Alonso-Mora, and O. Hilliges, “Real-time planning for automated multi-view drone cinematography,” *ACM Transactions on Graphics (TOG)*, vol. 36, no. 4, p. 132, 2017.
- [15] M. Ginesi, D. Meli, A. Roberti, N. Sansonetto, and P. Fiorini, “Autonomous task planning and situation awareness in robotic surgery,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [16] Q. Chen, J. Yu, M. Schwager, P. Abbeel, F. Shentu, and P. Wu, “Sarm: Stage-aware reward modeling for long horizon robot manipulation,” 2025. [Online]. Available: <https://arxiv.org/abs/2509.25358>
- [17] F. Faure, C. Duriez, H. Delingette, J. Allard, B. Gilles, S. Marchesseau, H. Talbot, H. Courtecuisse, G. Bousquet, I. Peterlik, and S. Cotin, *SOFA: A Multi-Model Framework for Interactive Physical Simulation*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012, pp. 283–321. [Online]. Available: https://doi.org/10.1007/8415_2012_125
- [18] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.